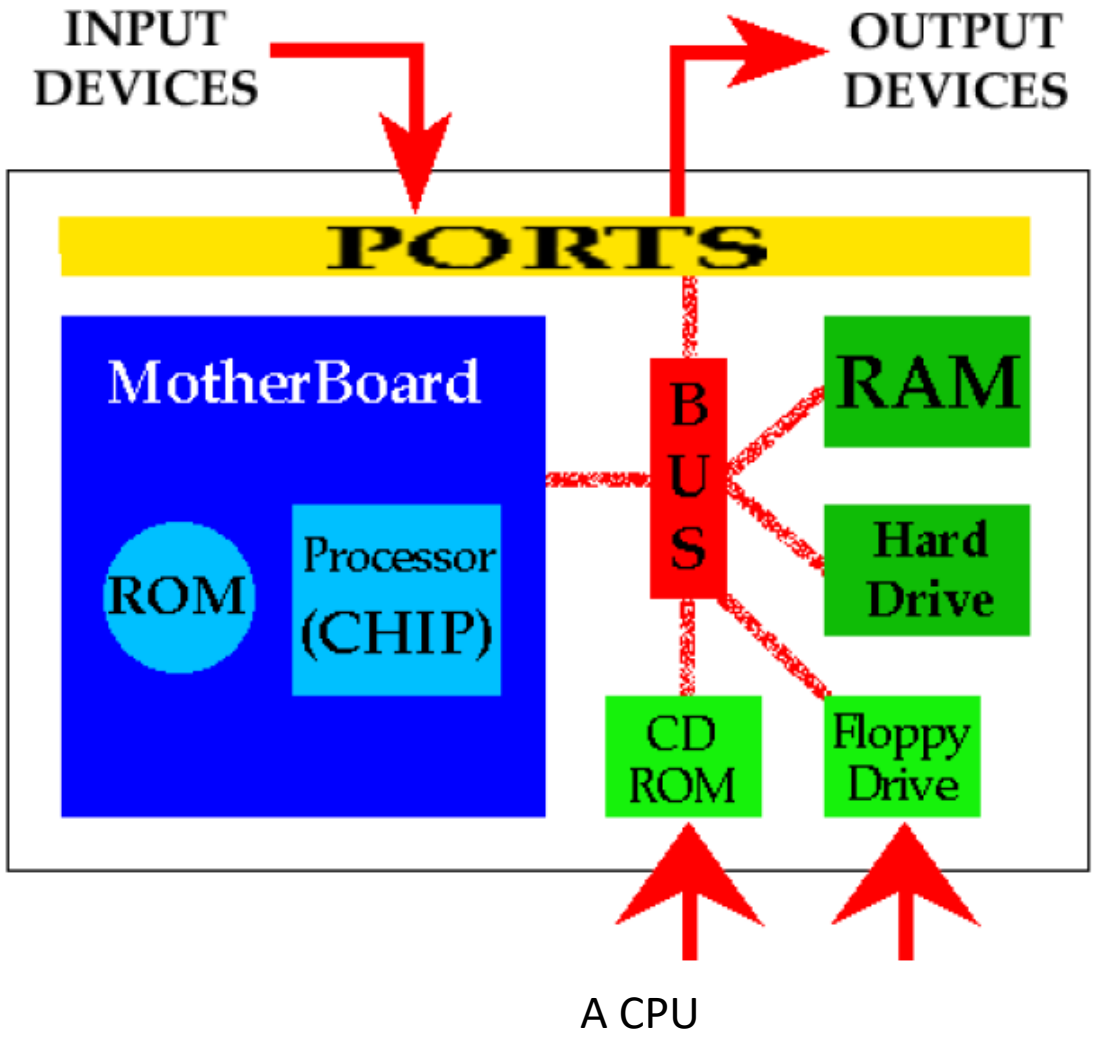
Architecture of a simple real computer

- Main memory
- Registers
- ALU
- Input/Output devices
- Disk
- Bus



Registers

- Much faster, costlier and smaller than main memory
- Registers are temporary storage areas for instructions or data.
- They are not a part of memory; rather they are special additional storage locations that offer the advantage of speed.

1. **Accumulator**
2. **Address register**
3. **Storage register**
4. General-purpose **register**

Main memory

- Memory is also known as primary storage, primary memory, main storage, internal storage, main memory, and RAM (Random Access Memory)
- Memory is the part of the computer that holds data and instructions for processing only temporally :

1. data is destroyed when the machine is turned off.
2. If more than one program is running, a single program can not lay exclusive claim to memory.
3. There may not be room in memory to hold the processed data

garbage value

CPU

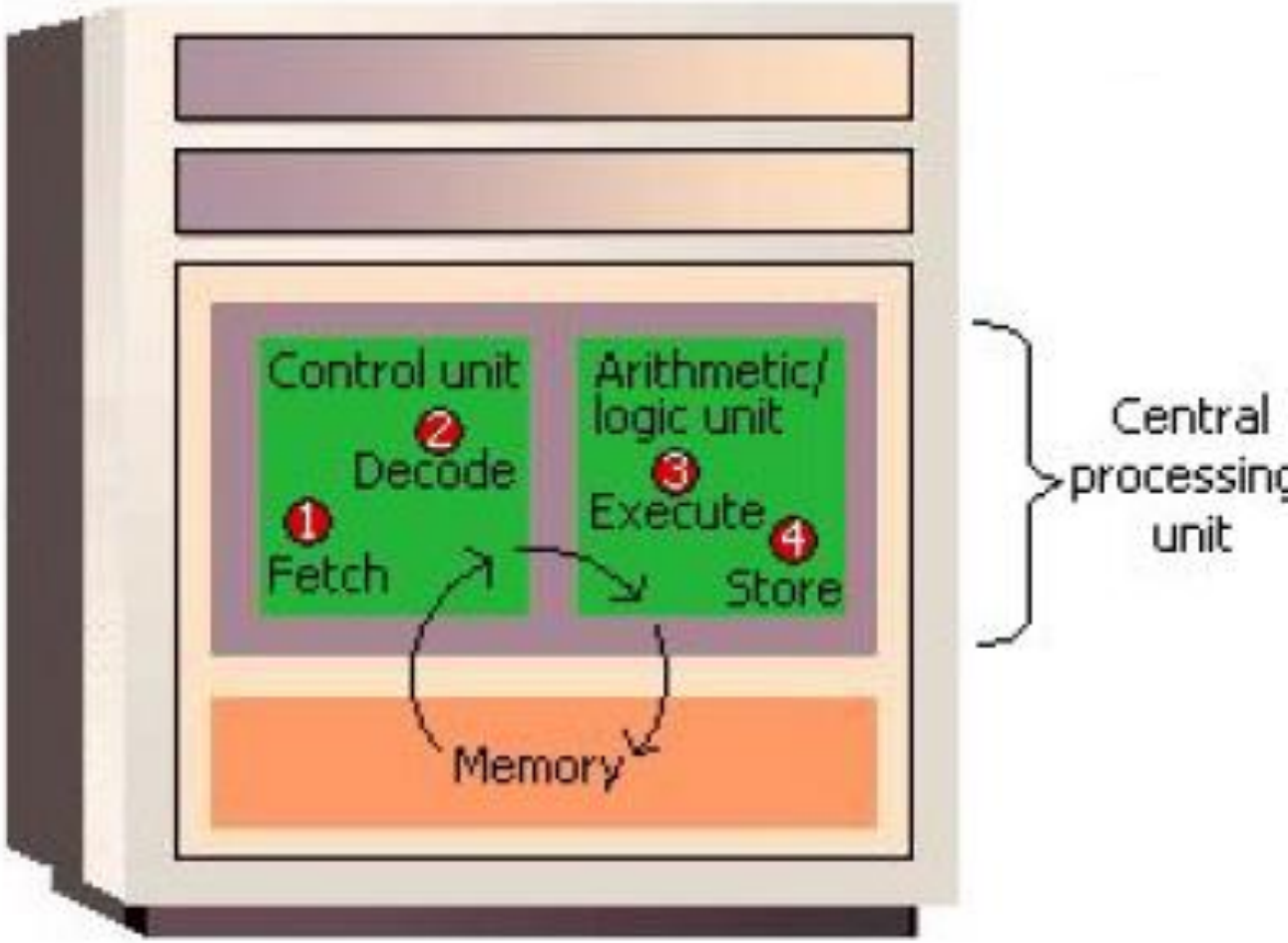
- **The Control Unit:** The control unit of the CPU contains circuitry that uses electrical signals to direct the entire computer system to carry out, or execute, stored program instructions.

- **The Arithmetic/Logic Unit:**
- The arithmetic/logic unit (ALU) contains the electronic circuitry that executes all arithmetic and logical operations.
- Takes operands from registers, stores result in registers

Machine Cycle

The central processing unit performs the following four steps for each instruction:

- Fetch
- Decode
- Execute
- Store



Bus

- Highways for transferring information between different parts
- Address bus, data bus, I/O bus, external device bus (Universal Serial Bus)

Basic Programming Languages

- There are three basic types of programming languages.

1. Machine level language
2. Assembly language
3. High-level languages

Machine language

- Machine language is just a sequence of zeroes and ones.
- Different combinations of zeroes and ones mean different things and sends different instruction to the computer.
- **Advantage:** This is the only language that a computer can understand directly.
- **Drawback**: This is not very easy for humans to be able to either read it or write it. It will be hard to get any developers to utilize this language.

Assembly language

- Assembly language is the oldest form of non-machine language.
- It is a symbolic representation of machine language.
- It typically uses a one to one correspondence between the machine and high level language
- Assembler: Translate the assembly language into machine language.
- **Advantage:** Since assembly language uses easy to recognize codes, it makes it a lot easier for people to understand.
- **Disadvantage:** Assembly language requires detailed knowledge of internal *computer architecture*

Example:

```
LOAD    BASEPAY
ADD     OVERPAY
STORE   GROSSPAY
```

A Machine Language Program Fragment and Its Assembly Language Equivalent

Memory Addresses	Machine Language Instructions	Assembly Language Instructions
00000000	00000000	CLA
00000001	00010101	ADD A
00000010	00010110	ADD B
00000011	00110101	STA A
00000100	01110111	HLT
00000101	?	A ?
00000110	?	B ?

High-level languages

- Words and commands (instead of just symbols and numbers) were being used.
- Machine independent
- These languages therefore, had syntax that was much easier to understand

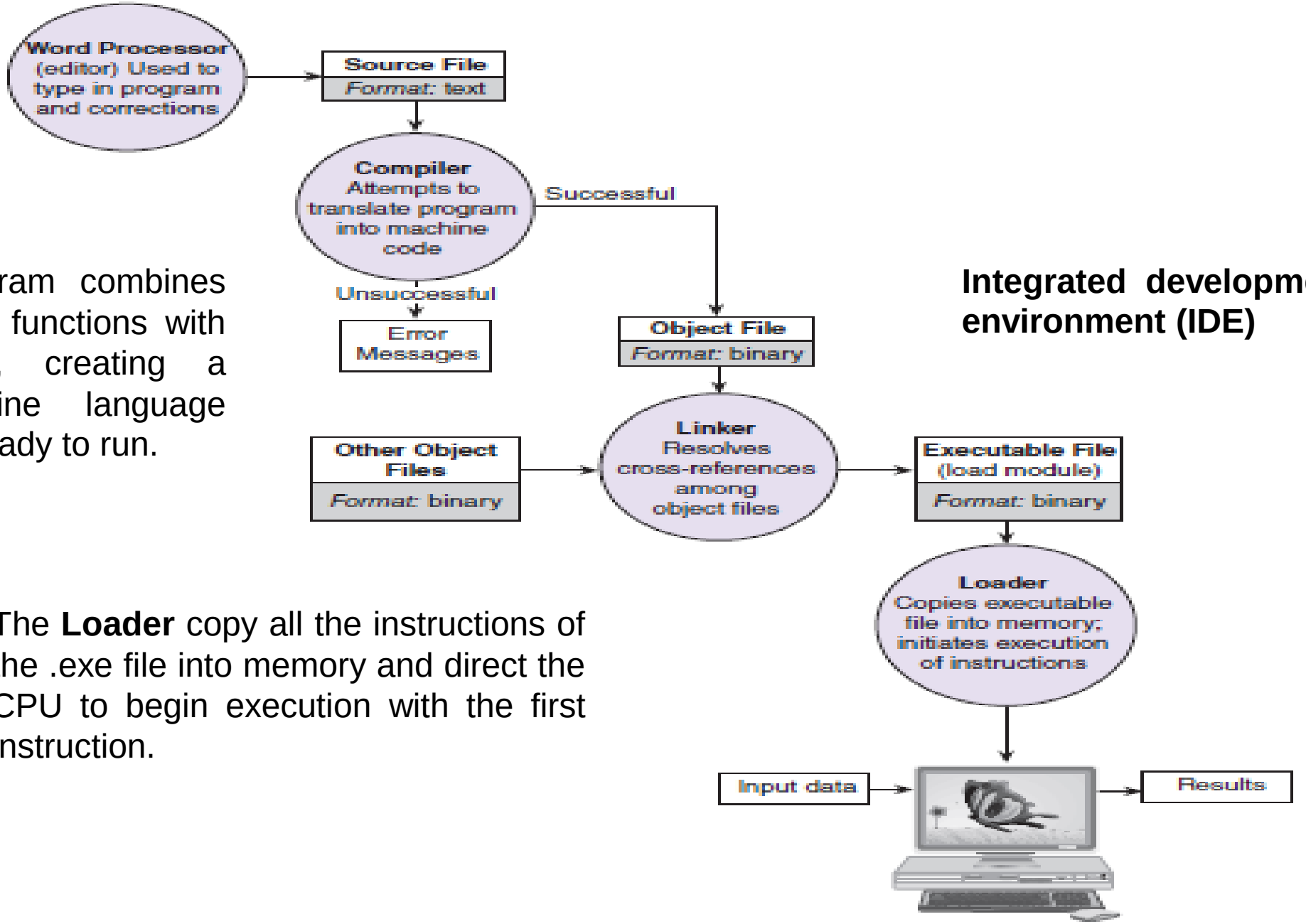
  a=b+c;

- Procedural Language
  - Fortran
  - Cobol
  - Basic
  - C
  - Pascal
- Object-Oriented Language
  - C++
  - Java
  - C#
- Functional Language
  - Lisp
- Logic Language
  - Prolog

Translation

- A program written in a high level language (source code) can only be run in its machine code equivalent format.
- There are two ways of achieving this:
  - Compilation, and Interpretation.

Entering, Translating, and Running a High-Level Language Program



The **linker** program combines the prefabricated functions with the object file, creating a complete machine language program that is ready to run.

Integrated development environment (IDE)

The **Loader** copy all the instructions of the .exe file into memory and direct the CPU to begin execution with the first instruction.

The C Programming Language

- C evolved from BCPL and B.
- BCPL (1967, Martin Richards)

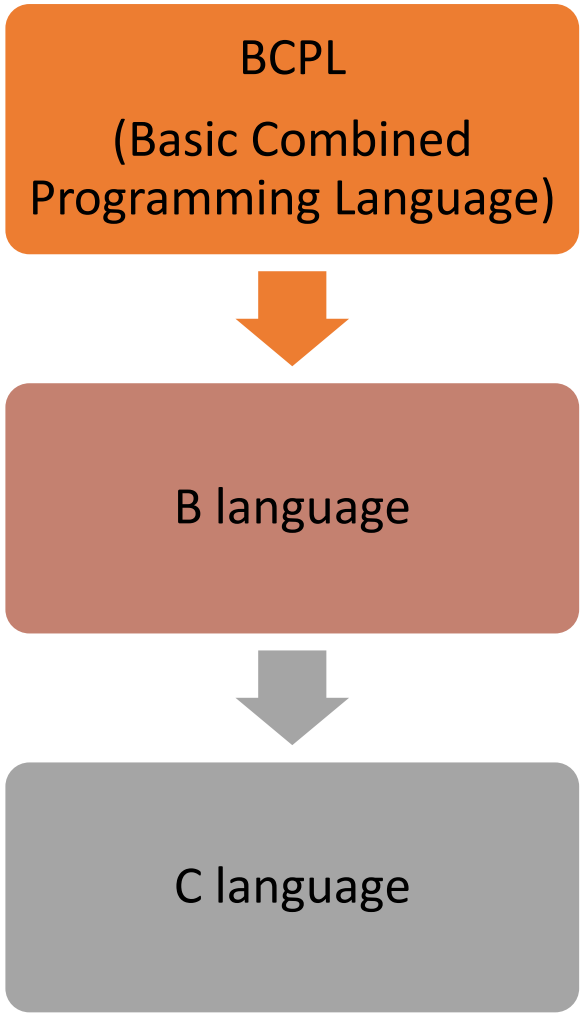
  Language for writing operating-systems software and compilers.

- B (1970, Ken Thompson)

  To create early versions of the UNIX operating system at Bell Laboratories.

- Both BCPL and B were typeless languages -

  The only data type is machine word and access to other kinds of objects is by special operators or function calls.



The C Programming Language (cont…)

- Evolved from B by Dennis Ritchie at Bell Laboratories and was originally implemented in 1972.
- Initially known as the development language of the UNIX operating system.
- Many of todays leading operating systems are written in C and/or C++.
- Hardware independent.
- Widely used to develop systems that demand performance

E.g. Operating systems, embedded systems, real-time systems and communications systems

The First C Program

Before we begin with our first C program do remember the following rules that are applicable to all C programs:

(a) Each instruction in a C program is written as a separate statement. Therefore a complete C program would comprise of a series of statements.

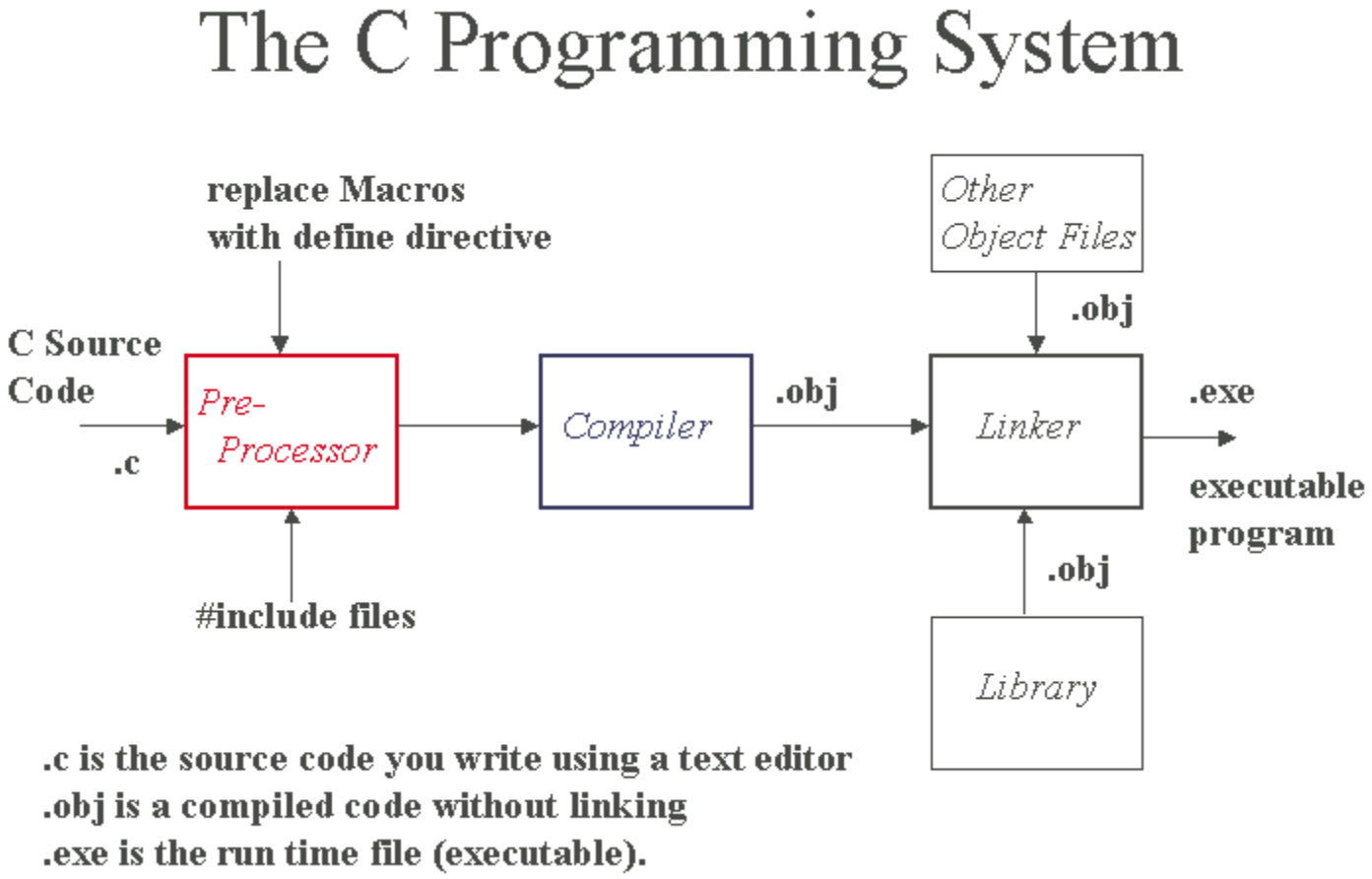
(b) The statements in a program must appear in the same order in which we wish them to be executed

(c) Blank spaces may be inserted between two words to improve the readability of the statement.

(d) All statements are entered in small case letters. C has no specific rules for the position at which a statement is to be written. That's why it is often called a free-form language.

(e) Every C statement must end with a ';'. Thus ; acts as a statement terminator.

Example of a C Program

```
/* Print “Hello World” */
/* Author ABC Date: 18/07/2018 */
#include <stdio.h>
int main( )
{
        printf ( “Hello World!” ) ;
        return 0;
}
```

Compilation of a C Program